

Lista zadań nr 3

Listy, słowniki i zbiory składane. Zadania symulacyjne.

Zadanie 1 Wykorzystując listy składane rozwiąż następujące zadania (utwórz odpowiednie listy):

- a) Utwórz 20-elementową listę L losowych liczb całkowitych z zakresu od 0 do 100.
- b) Utwórz listę kwadratów liczb z listy L.
- c) Utwórz listę liczb parzystych z listy L.
- d) Utwórz listę liczb z listy L, ale zapisanych w zapisie binarnych (np. liczba 5 powinna być konwertowana na "101" (napis)).
- e) Utwórz listę tylko tych liczb które należą do przedziału [20, 50] z listy L.
- f) Utwórz 20-elementową listę S1 losowych ciągów tekstowych składających się z małych liter alfabetu łacińskiego o losowej długości z zakresu 3 - 30.
- g) Utwórz listę ciągów tekstowych, które nie zawierają samogłosek z listy S1.
- h) Utwórz 20-elementową listę S2 losowych ciągów tekstowych składających się z małych liter alfabetu łacińskiego i cyfr o losowej długości z zakresu 3 - 30.
- i) Na podstawie listy S2 utwórz listę z napisów oczyszczonych z cyfr.
- j) Na podstawie listy S1 utwórz listę z napisów oczyszczonych z liter, następnie przekształć je na liczby całkowite (pamiętaj o konieczności odrzucenia łańcuchów pustych).
- k) Utwórz 20-elementową listę K losowych krotek losowych liczb całkowitych z zakresu od 0 do 100 o losowej długości z zakresu 1 - 6.
- l) Z podanej listy K utwórz listę krotek dwuelementowych. Odrzuć krótsze krotki, a dłuższe przytnij do pierwszych dwóch elementów.
- m) Z podanej listy K utwórz listę krotek, których suma elementów jest nieparzysta.
- n) Z podanej listy K utwórz listę krotek, których wszystkie elementy są podzielne przez 3.
- o) Z podanej listy K utwórz listę krotek, w których istnieje element podzielny przez 3.

Zadanie 2 Dana jest lista

```
vec = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Utwórz listę, w której wszystkie elementy list zagnieżdżonych w `vec` występują na pierwszym poziomie, tj. `[1, 2, 3, 4, 5, 6, 7, 8, 9]`. Przedstaw dwa rozwiązania: z zagnieżdżonymi pętlami `for` oraz z wykorzystaniem listy składanej.

Zadanie 3 Dana jest lista reprezentująca macierz:

```
matrix = [  
    [1, 2, 3, 4],  
    [5, 6, 7, 8],  
    [9, 10, 11, 12],  
]
```

Napisz program, który utworzy na jej podstawie listę reprezentującą macierz transponowaną:

```
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

Wykorzystaj zarówno listy składane, jak i zagnieżdżone pętle (napisz co najmniej dwa różne programy). Napisz funkcję, która wyświetla macierz jak tablice (podobnie jak robimy to na matematyce) - argumentem wywołania powinna być lista reprezentująca macierz.

Zadanie 4 Napisz funkcję, która dla podanego jej jako argument ciągu tekstowego zliczy liczbę wystąpień poszczególnych liter w tym ciągu (inne znaki nie powinny być zliczane). Małe i duże litery powinny być traktowane jako te same litery. Wykorzystaj słownik by przechowywać informację o liczbie wystąpień danych liter w podanym tekście. Następnie wykorzystaj typ `Counter` z modułu `collections` aby wykonać to zadanie jeszcze prościej. Wyświetl informację o liczbie wystąpień w danej literze zgodnie z porządkiem alfabetycznym.

Zadanie 5 Wyobraź sobie, że piszesz pewną grę RPG w wyniku zabicia przeciwnika otrzymujesz listę łupów - być może na liście jest wiele łupów tego samego typu. Napisz funkcję, która konwertuje listę znalezionych łupów na słownik (lub inny obiekt typu odwzorowującego) reprezentujący twój ekwipunek, gdzie dla każdego przedmiotu podana jest jego ilość. Następnie napisz funkcję, która wyświetli zawartość ekwipunku w kolejności.

Zadanie 6 Napisz funkcję która będzie zwracała wartość True jeżeli przekazany jej ciąg tekstowy będzie zawierał zrównoważoną liczbę nawiasów tj. każdy otwarty nawias powinien być zamknięty. W przeciwnym wypadku funkcja powinna zwracać wartość False. Uwzględnij cztery rodzaje nawiasów: (), [], {} oraz <>. Wykorzystaj słowniki.

Zadanie 7 Napisz program, który pobierze od użytkownika napis. Program powinien wyświetlić informację, z ilu różnych znaków składa się dany wyraz. Zadanie powinna realizować funkcja.

Zadanie 8 Wykorzystaj zbiory oraz działania na nich celem napisania programu, który pobierze od użytkownika dwa napisy, po czym wyświetli (wraz z odpowiednim komunikatem):

- wszystkie znaki, które znajdują się w jednym lub drugim napisie;
- wszystkie znaki, które występują w jednym napisie, a nie występują w drugim (osobno);
- znaki, które znajdują się zarówno w jednym, jak i w drugim napisie.

Zadanie 9 Masz dwie listy:

```
keys = ['a', 'b', 'c', 'd']  
values = [1, 2, 3, 4]
```

Wykonaj następujące operacje w jednym zadaniu:

1. Za pomocą słownika składnego, połącz klucze i wartości z powyższych list, aby utworzyć słownik.
2. Dodaj do słownika nowy klucz 'e' z wartością będącą sumą wszystkich wartości w słowniku.
3. Zaktualizuj słownik tak, aby każda wartość była podniesiona do kwadratu (po-
nownie używając słownika składnego).
4. Na koniec odfiltruj tylko te klucze, których wartości są większe niż 10, i utwórz nowy słownik.

Zadanie 10 Masz następujący ciąg znaków:

```
text = "Trzeba być zawsze tylko sobą. Koń bez ułana jest  
zawsze koniem. Ułan bez konia tylko człowiekiem."
```

Wykonaj następujące operacje w jednym zadaniu:

1. Utwórz słownik składany, w którym kluczami są unikalne słowa z tekstu, a wartościami liczba wystąpień każdego słowa (ignorując wielkość liter). W tym samym słowniku, jako wartości, przechowuj również długości słów w osobnym podkluczu, np. {'trzeba': {'count': 1, 'length': 6}, ...}.
2. Usuń z tego słownika wszystkie słowa, które występują tylko raz i mają mniej niż 5 znaków.
3. Posortuj słownik według długości słów (malejąco).

Zadanie 11 Masz następujące dane:

```
L = list(range(1, 21))  
text = "Ta kobieta była jak moneta, szła z ręki do ręki,  
ale nie straciła na wartości."
```

Wykonaj następujące kroki używając zbiorów składanych:

1. Utwórz zbiór wszystkich liczb z listy L, które są podzielne przez 3 lub przez 5.
2. Utwórz zbiór unikalnych długości słów w powyższym tekście (ignorując wielkość liter).
3. Utwórz nowy zbiór, który zawiera liczby wspólne dla zbioru z punktu 1 i długości słów z punktu 2.
4. Dodaj do wyniku z punktu 3 liczbę 42, ale tylko jeśli nie jest już w zbiorze. Sprawdź, czy wynikowy zbiór jest podzbiorem zbioru {1, 2, 3, 5, 10, 15, 20, 42} i wyświetl wynik.

Zadanie 12 Masz dwa ciągi znaków:

```
text1 = "Python to język programowania. Python to język  
        obiektowy."  
text2 = "Programowanie w języku Python jest bardzo popularne.  
        Python jest jest językiem interpretowanym."
```

Wykonaj następujące operacje:

1. Utwórz zbiory unikalnych słów z obu tekstów, ignorując wielkość liter.
2. Znajdź zbiór wspólnych słów w obu tekstach.
3. Utwórz zbiór słów występujących tylko w jednym z tekstów.
4. Utwórz zbiór długości wspólnych słów z punktu 2.
5. Posortuj wartości zbioru z punktu 4 i wyświetl je jako listę.

Zadanie 13 Napisz program *"Prosta gra w statki"*, który symuluje uproszczoną wersję gry w statki. Program powinien tworzyć planszę (np. 10×10), na której losowo rozmieszczone będą jednomasztowe statki (np. 5 statków).

Użytkownik powinien mieć możliwość podawania kolejnych współrzędnych pól (np. w formacie liczbowym: wiersz, kolumna), próbując zatopić wszystkie statki. Program powinien wyświetlać aktualny stan planszy, pokazując pola trafione, nietrafione oraz nieodkryte.

Zastosuj następujące oznaczenia:

- nieodkryte pole – symbol kropki ".",
- trafiony statek – symbol "X",
- nietrafione pole – symbol "0".

Program powinien zawierać co najmniej następujące funkcje:

- `set_ships()` – losuje współrzędne jednomasztowych statków i zwraca zbiór tych współrzędnych.
- `print_board(board)` – wyświetla aktualny stan planszy.
- `game()` – realizuje całą rozgrywkę aż do momentu zatopienia wszystkich statków, komunikując się z użytkownikiem.

Po zatopieniu wszystkich statków program powinien wyświetlić komunikat o zakończeniu gry oraz liczbę ruchów wykonanych przez gracza.

Zadanie 14 Napisz program symulujący działanie „magicznej kuli” – zabawki wyświetlającej losowe odpowiedzi na zadawane pytania. Program powinien przechowywać listę 12 możliwych odpowiedzi bezpośrednio w kodzie, np.:

```
answers = [  
    "Nie sędę.",  
    "Oczywiście, że tak!",  
    "Nie wiem.",  
    "Zapytaj ponownie później.",  
    "Na pewno nie!",  
    "Jest to prawdopodobne.",  
    "Wszystko wskazuje na tak.",  
    "Możliwe.",  
    "Wątpliwe.",  
    "Bez wątpienia!",  
    "Nie licz na to.",  
    "To pewne!"  
]
```

Program powinien umożliwiać użytkownikowi wielokrotne zadawanie pytań. Po wpisaniu każdego pytania program powinien wyświetlać jedną losowo wybraną odpowiedź z listy. Gra powinna trwać do momentu, aż użytkownik zdecyduje się ją zakończyć (np. wpisując puste pytanie lub słowo koniec).

Program powinien zawierać przynajmniej następujące funkcje:

- `get_random_answer(answers)` – losowo wybiera i zwraca jedną odpowiedź z listy.
- `play_game(answers)` – przeprowadza całą rozgrywkę, obsługując pytania użytkownika aż do zakończenia gry.

Program powinien być czytelny, przejrzysty oraz wygodny w obsłudze.

Zadanie 15 Jednoręki bandyta to maszyna hazardowa, do której gracz wrzuca monety, po czym pociąga za dźwignię (lub naciska przycisk). Maszyna wyświetla serię losowych symboli. Jeśli co najmniej dwa symbole do siebie pasują, gracz wygrywa określoną sumę pieniędzy.

Zaprojektuj program, który będzie symulował działanie jednorękiego bandyty. Po uruchomieniu program powinien działać według następujących zasad:

1. Program prosi gracza o wrzucenie do maszyny dowolnej dodatniej sumy pieniędzy.
2. Następnie program trzykrotnie losuje i wyświetla słowo z poniższej listy:
wiśnie, pomarańcze, śliwki, dzwonki, melony, sztabki
3. Program określa wynik gry w następujący sposób:
 - jeśli żadne z wyświetlonych słów się nie powtarza, wygrana wynosi 0;
 - jeśli pasują do siebie dwa słowa, wygrana wynosi dwukrotność sumy wrzuconej do maszyny;
 - jeśli pasują wszystkie trzy słowa, wygrana wynosi trzykrotność sumy wrzuconej do maszyny.

Program powinien wyświetlić odpowiedni komunikat o wygranej kwocie.

4. Po zakończonej rundzie program pyta gracza, czy chce zagrać ponownie. Jeśli tak, kroki od 1 do 3 należy powtórzyć. Jeśli nie, program powinien zakończyć działanie, wyświetlając sumaryczną kwotę wszystkich wrzuconych do maszyny pieniędzy oraz łączną kwotę wszystkich wygranych.

Program powinien zawierać co najmniej następujące funkcje:

- `draw_symbols()` – losuje trzy symbole z listy i zwraca je.
- `calculate_winnings(bet, symbols)` – oblicza wygraną na podstawie wrzuconej kwoty i wylosowanych symboli.
- `play_round()` – obsługuje pojedynczą rundę gry.
- `main()` – obsługuje przebieg całej gry oraz sumowanie wyników wszystkich rund.

Program powinien być czytelny, przejrzysty oraz intuicyjny w obsłudze.

Zadanie 16 Napisz program, który generuje losową tablicę dwuwymiarową $n \times n$ (rozmiar n powinien być ustalany przez użytkownika z zakresu od 10 do 20), wypełnioną losowymi cyframi od 0 do 9. Program powinien wyszukać wszystkie wystąpienia czterech takich samych cyfr ułożonych obok siebie w kolumnie, wierszu lub po przekątnej.

Program powinien wyświetlić na ekranie wygenerowaną tablicę, przy czym znalezione "czwórki" powinny być wyraźnie zaznaczone (np. otoczone nawiasami, oznaczone gwiazdką '*' lub innym czytelnym symbolem).

Program powinien umożliwiać wielokrotne przeprowadzenie symulacji (użytkownik decyduje, kiedy zakończyć działanie programu).

Program powinien zawierać co najmniej następujące funkcje:

- `generate_random_grid(n)` – tworzy losową tablicę o wymiarach $n \times n$.
- `display_grid(grid)` – wyświetla tablicę na ekranie w czytelnej postaci, z zaznaczonymi „czwórkami”.
- `find_fours(grid)` – wyszukuje wszystkie wystąpienia czwórek jednakowych cyfr ułożonych poziomo, pionowo oraz ukośnie i zwraca ich współrzędne.
- `simulation()` – umożliwia wielokrotne przeprowadzenie symulacji według decyzji użytkownika.

Program powinien być czytelny oraz wygodny w użytkowaniu.

Zadanie 17 Napisz program symulujący tzw. *problem kolekcjonera kuponów*.

Opis podstawowego wariantu problemu:

1. Dysponujemy n urnami, gdzie wartość n wybiera użytkownik z zakresu 2–100.
2. Kolejno wrzucamy kule do urn, przy czym każda urna jest wybierana z jednakowym prawdopodobieństwem, a kolejne wybory są niezależne.
3. Proces kontynuujemy, aż w każdej z n urn znajdzie się co najmniej jedna kula.

Celem symulacji jest wyznaczenie liczby rzutów T_n potrzebnych do tego, aby każda urna zawierała przynajmniej jedną kulę. Program powinien umożliwiać wielokrotne przeprowadzenie symulacji (liczbę powtórzeń określa użytkownik).

Program powinien składać się z następujących funkcji:

- `single_simulation(n)` – wykonuje jedną symulację dla podanej liczby urn n i zwraca liczbę potrzebnych rzutów.
- `multiple_simulations(n, num_simulations)` – przeprowadza wielokrotną symulację i zbiera wyniki rzutów.
- `show_summary(results)` – prezentuje wyniki symulacji (np. średnią liczbę potrzebnych rzutów).

Program powinien umożliwić wygodne ustawienie liczby urn oraz liczby symulacji, a także przejrzyste prezentować wyniki.

Zadanie 18 Napisz program symulujący grę „Życie” (ang. *Game of Life*). Program powinien pokazać na ekranie kilka kolejnych generacji (cykli życia).

Opis: Dana jest tablica o rozmiarze $n \times n$ reprezentująca komórki. Tablica powinna być losowo wypełniona wartościami 0 lub 1, gdzie 0 oznacza martwą komórkę, a 1 – komórkę żywą. Każda komórka ma ośmiu sąsiadów (z lewej, prawej, góry, dołu oraz po przekątnych). Stan komórki (żywa lub martwa) zmienia się według następujących reguł:

- a) Martwa komórka staje się żywa, gdy ma dokładnie trzech żywych sąsiadów.
- b) Żywa komórka pozostaje żywa, gdy ma 2 lub 3 żywych sąsiadów.
- c) Żywa komórka umiera, gdy ma mniej niż 2 sąsiadów (z samotności).
- d) Żywa komórka umiera, gdy ma więcej niż 4 sąsiadów (z przeludnienia).

Tablica powinna być traktowana jako dwuperiodyczna (w topologii torusa), co oznacza, że komórki na brzegach tablicy sąsiadują z komórkami z przeciwległych brzegów. Program powinien zawierać przynajmniej następujące funkcje:

- `random_grid(n)` – tworzy i zwraca losowo wygenerowaną tablicę początkową o wymiarach $n \times n$.
- `display(grid)` – wyświetla aktualny stan tablicy `grid`.
- `count_neighbors(grid, row, col)` – zwraca liczbę żywych sąsiadów wskazanej komórki, uwzględniając periodyczność.
- `next_generation(grid)` – generuje nową tablicę na podstawie bieżącej generacji zgodnie z zasadami gry.

Program powinien umożliwiać wygodne ustawienie rozmiaru tablicy (n).

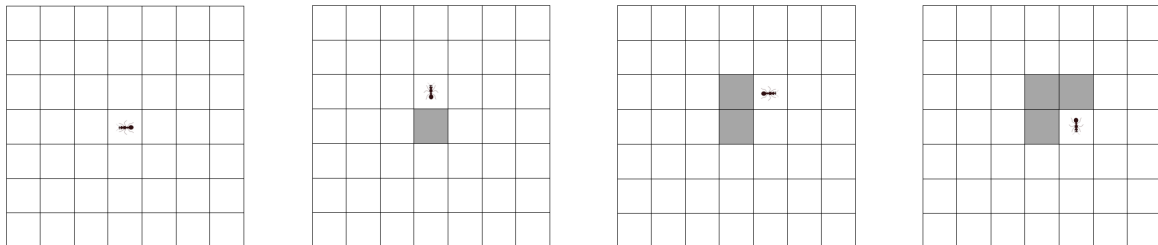
Zadanie 19 Mrówka Langtona to prosty automat komórkowy wymyślony i opisany przez Chrisa Langtona. Mrówka porusza się według określonych zasad po dwuwymiarowej siatce pól, zmieniając ich kolory na przeciwne.

Początkowo mrówka znajduje się na centralnym polu tablicy (np. o rozmiarze 20×20), zwrócona jest na zachód. Dalej mrówka radzi sobie sama, według następujących reguł:

- Jeśli mrówka znajduje się na polu białym (oznaczanym spacją ' '), obraca się w prawo o 90° , zmienia kolor pola na szary (oznaczany znakiem '#') i przechodzi na następną komórkę.

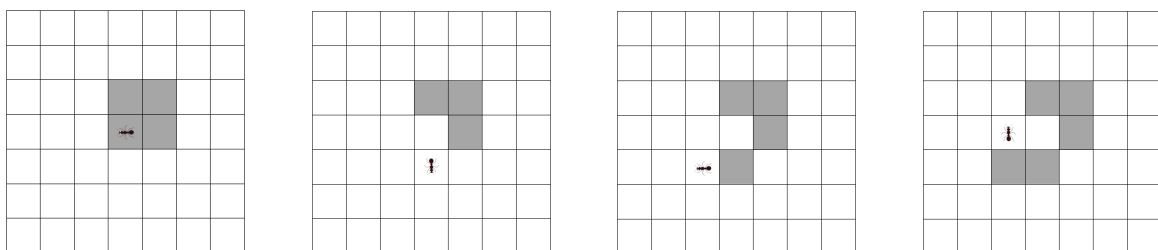
- Jeśli mrówka znajduje się na polu szarym ('#'), obraca się w lewo o 90°, zmienia kolor pola na biały (spacja ' ') i przechodzi na następną komórkę.
- Mrówka porusza się po tablicy dwuperiodycznej (jak w grze Snake), tzn. wychodząc poza jeden brzeg planszy, pojawia się na przeciwnym.

Poniżej znajduje się symulacja kilku pierwszych ruchów mrówki Langtona:



Na pierwszym rysunku (od lewej) widzimy mrówkę w pozycji startowej. Ponieważ pole jest białe, mrówka obraca się w prawo, zmienia kolor pola na szary i przesuwa się do przodu. Analogicznie mrówka zachowuje się w kolejnych krokach.

Kolejne kroki mrówki przedstawiono poniżej:



Zaimplementuj w języku Python symulację działania mrówki Langtona dla n kroków, gdzie n ustala użytkownik.

Program powinien zawierać przynajmniej następujące funkcje:

- `create_grid(size)` – tworzy i zwraca tablicę o podanym rozmiarze.
- `display_grid(grid)` – wyświetla aktualny stan tablicy.
- `move_ant(grid, ant_position, ant_direction)` – wykonuje jeden krok mrówki zgodnie z powyższymi zasadami i zwraca nową pozycję oraz kierunek mrówki.
- `simulate(grid_size, steps)` – wykonuje całą symulację mrówki Langtona przez zadaną liczbę kroków.

Wynikiem symulacji powinien być końcowy stan planszy. Opcjonalnie program może wyświetlać stan planszy po każdym kroku symulacji.

Zadanie 20 Napisz program symulujący spadanie kulki po desce Galtona. Program powinien umożliwiać użytkownikowi wybór liczby szeregów kołków (od 1 do 12). Kulka, napotykając kołek, z jednakowym prawdopodobieństwem może spaść w lewo lub w prawo. Wynikiem pojedynczej symulacji powinien być numer pojemnika, do którego wpadła kulka.

Program powinien umożliwiać wielokrotne przeprowadzanie symulacji (użytkownik powinien podawać liczbę prób), a na koniec wyświetlić podsumowanie wyników w postaci tekstowego histogramu (np. używając gwiazdek '*' do oznaczenia liczby trafień).

W programie powinny znaleźć się przynajmniej następujące funkcje:

- `single_simulation(n)` – przeprowadza pojedynczą symulację dla deski z n rzędami kołków, zwracając numer pojemnika, do którego wpadła kulka.
- `run_simulations(n, trials)` – przeprowadza `trials` symulacji i zbiera wyniki.
- `show_results(results)` – wyświetla tekstowy histogram wyników symulacji bez użycia zewnętrznych bibliotek graficznych.

Program powinien umożliwiać wygodny wybór parametrów symulacji (liczby rzędów kołków oraz liczby prób).

Zadanie 21 Napisz program symulujący grę Penneya dla podanych par serii:

- para 1: 00R0 kontra 0R00
- para 2: 0R00 kontra R000
- para 3: R000 kontra 00R0

Dla każdej z wymienionych par program powinien przeprowadzić dokładnie 1000 symulacji gry i przedstawić zestawienie wyników (liczbę zwycięstw każdego ciągu).

Opis gry:

Gra Penneya polega na tym, że dwaj gracze wybierają serie orłów (O) i reszek (R) o jednakowej długości (np. 00R0 i R000). Następnie rzucamy monetą, zapisując wynik każdego rzutu (litery O i R). Rzuty kontynuujemy, aż wygenerowana sekwencja zakończy się serią wybraną przez jednego z graczy. Wtedy ten gracz wygrywa grę.

Przykład:

- gracz1: **ORRO**
- gracz2: **R000**

Rzuty mogą wyglądać następująco: OROR00R0R000

W tym przypadku wygrywa gracz2, a do zakończenia gry potrzebnych było 12 rzutów.

Program powinien zawierać przynajmniej następujące funkcje:

- `single_game(seq1, seq2)` – przeprowadza jedną symulację gry dla dwóch podanych serii `seq1, seq2`. Zwraca informację o zwycięzcy oraz liczbie rzutów.
- `simulate_games(seq1, seq2, num_games)` – przeprowadza `num_games` symulacji, zbiera wyniki gier i zlicza liczbę zwycięstw każdego z graczy.
- `show_results(results)` – wyświetla zestawienie wyników symulacji (np. liczbę zwycięstw każdego ciągu).

Program powinien wyświetlić przejrzyste podsumowanie dla każdej z trzech wymienionych par serii.